Retrieval-Augmented Generation: Original Formulation and Modern Practice

1 Original RAG (Lewis et al., 2020)

Retriever (DPR). A document z is scored against the query x with the dot product of two BERT encoders:

$$p_\eta(z \mid x) \;\propto\; \exp\big(\mathbf{d}(z)^\top \mathbf{q}(x)\big), \qquad \mathbf{d}(z) = \mathrm{BERT}_d(z), \;\; \mathbf{q}(x) = \mathrm{BERT}_q(x). \tag{1}$$

Only the top-k documents are kept, found with maximum inner product search (MIPS) via FAISS.

Generator (BART). The generator $p_\theta(y_i \mid x, z, y_{1:i-1})$ produces the answer token by token, conditioned on the query and one document. The retrieved document z is treated as a latent variable and marginalized out. There are two variants.

RAG-Sequence uses the same document for the whole answer:

$$p(y \mid x) \;\approx\; \sum_{z \in \text{top-}k} p_\eta(z \mid x) \prod_{i=1}^{N} p_\theta(y_i \mid x, z, y_{1:i-1}). \tag{2}$$

RAG-Token can use a different document for each token:

$$p(y \mid x) \;\approx\; \prod_{i=1}^{N} \sum_{z \in \text{top-}k} p_\eta(z \mid x)\, p_\theta(y_i \mid x, z, y_{1:i-1}). \tag{3}$$

Training. Training minimizes the negative marginal log-likelihood over the training pairs (x_j, y_j):

$$\mathcal{L} = -\sum_j \log p(y_j \mid x_j). \tag{4}$$

The document encoder BERT_d is kept frozen, so the index never needs rebuilding. Only BERT_q and BART are updated.

2 Modern Practical RAG

Without training, RAG reduces to three steps.

$$\textbf{1. Embed:} \quad \mathbf{e}_q = E(x), \qquad \mathbf{e}_z = E(z) \;\; \forall z \in \mathcal{D}. \tag{5}$$

$$\textbf{2. Retrieve:} \quad Z_k = \underset{z \in \mathcal{D}}{\text{top-}k} \cos(\mathbf{e}_q, \mathbf{e}_z) = \underset{z \in \mathcal{D}}{\text{top-}k} \frac{\mathbf{e}_q^\top \mathbf{e}_z}{\|\mathbf{e}_q\| \, \|\mathbf{e}_z\|}. \tag{6}$$

$$\textbf{3. Generate:} \quad y = \mathrm{LLM}\big(\mathrm{prompt}(x, Z_k)\big). \tag{7}$$

Step 3 is the in-context version of the paper's marginalization. Instead of summing probabilities over documents, the k documents are concatenated into the prompt and the LLM decides which ones to use.

3 From the Paper to Modern RAG

Modern RAG keeps the *idea* of Lewis et al. (retrieve from an external, non-parametric memory, then generate), but not their equations or training. The modern recipe is described most directly by two later papers.

In-Context RALM (Ram et al., 2023). Retrieved text is prepended to the input of a *frozen* language model, with no training at all. This is exactly the three-step pipeline of Section 2:

$$p(y \mid x) \;=\; \prod_{i=1}^{N} p_\theta\big(y_i \mid [\,z_1; \ldots; z_k; x\,],\, y_{1:i-1}\big), \qquad z_1, \ldots, z_k \in Z_k, \tag{8}$$

where $[\,\cdot\,;\,\cdot\,]$ is concatenation.

REPLUG (Shi et al., 2023). It treats the LLM as a black box and ensembles over the retrieved documents. This is RAG-Sequence (2) applied at inference time, with no gradient updates:

$$p(y \mid x) \;=\; \sum_{z \in Z_k} \lambda(z, x)\, p_{\text{LLM}}\big(y \mid [\,z; x\,]\big), \qquad \lambda(z, x) = \frac{\exp\big(\cos(\mathbf{e}_z, \mathbf{e}_q)\big)}{\sum_{z' \in Z_k} \exp\big(\cos(\mathbf{e}_{z'}, \mathbf{e}_q)\big)}. \tag{9}$$

	Original RAG (2020)	Modern RAG
Retriever	DPR, query encoder fine-tuned	Any pretrained embedding model, frozen
Generator	BART-large (400M), fine-tuned	Any LLM, frozen (local or API)
Combining docs	Marginalize $\sum_z p_\eta(z \mid x)\, p_\theta(\cdot)$	Concatenate Z_k into the prompt
Gradient updates	Yes, Eq. (4)	None

4 Fine-Tuning, LoRA and RAG

The key difference is *where* the knowledge lives: in the weights (parametric) or in the document store $\mathcal{D}$ (non-parametric).

Full fine-tuning. All weights θ are updated by gradient descent:

$$\theta^* = \arg\min_\theta \; -\sum_j \log p_\theta(y_j \mid x_j). \tag{10}$$

LoRA (Hu et al., 2021). The pretrained weight W_0 is frozen and only a low-rank update is trained:

$$W = W_0 + BA, \qquad B \in \mathbb{R}^{d \times r},\; A \in \mathbb{R}^{r \times m},\; r \ll \min(d, m), \tag{11}$$

$$(A^*, B^*) = \arg\min_{A,B} \; -\sum_j \log p_{W_0 + BA}(y_j \mid x_j). \tag{12}$$

Modern RAG. There is no arg min. The weights stay fixed and only the *input* changes:

$$y \sim p_\theta\big(\,\cdot \mid \text{prompt}(x, Z_k(x))\big), \qquad \theta \text{ fixed}. \tag{13}$$

To add or correct knowledge, you edit $\mathcal{D}$ and re-embed the changed chunks. Nothing is retrained.

5 Worked Example: Modern RAG, Step by Step

Step 0: Knowledge base and question. Four chunks of course information and one question:

z_1	"The Machine Learning exam takes place on 15 February."
z_2	"Machine Learning lectures are held in room H 104."
z_3	"The cafeteria serves pizza on Fridays."
z_4	"For the exam, one handwritten A4 cheat sheet is allowed."
x	"When is the Machine Learning exam?"

Step 1: Chunking. Real documents are long, so each one is split into chunks of L tokens with an overlap of o tokens. For a document with tokens $t_1, \ldots, t_T$, chunk j is

$$z_j = \big(t_{(j-1)(L-o)+1},\ \ldots,\ t_{(j-1)(L-o)+L}\big), \qquad j = 1, \ldots, \left\lceil \tfrac{T-o}{L-o} \right\rceil. \tag{14}$$

Typical values are $L \approx 200$–500 and $o \approx 10$–20% of L. Here each sentence is already one chunk.

Step 2: Embedding (offline, done once). An embedding model E maps text to a vector. Real models output 384–1024 dimensions. For hand calculation, use 3 dimensions roughly meaning (*exam*, *course*, *food*):

$$\mathbf{e}_{z_1} = \begin{pmatrix} 0.9 \\ 0.4 \\ 0.0 \end{pmatrix},\ \mathbf{e}_{z_2} = \begin{pmatrix} 0.1 \\ 0.9 \\ 0.1 \end{pmatrix},\ \mathbf{e}_{z_3} = \begin{pmatrix} 0.0 \\ 0.1 \\ 1.0 \end{pmatrix},\ \mathbf{e}_{z_4} = \begin{pmatrix} 0.7 \\ 0.2 \\ 0.1 \end{pmatrix}. \tag{15}$$

These vectors are stored in the index (e.g. a NumPy matrix, FAISS or Chroma).

Step 3: Embed the query (online).

$$\mathbf{e}_q = E(x) = \begin{pmatrix} 0.8 \\ 0.6 \\ 0.0 \end{pmatrix}, \qquad \|\mathbf{e}_q\| = \sqrt{0.64 + 0.36} = 1. \tag{16}$$

Step 4: Similarity. Cosine similarity $\cos(\mathbf{e}_q, \mathbf{e}_z) = \mathbf{e}_q^\top \mathbf{e}_z \,/\, (\|\mathbf{e}_q\|\,\|\mathbf{e}_z\|)$:

Chunk	$\mathbf{e}_q^\top \mathbf{e}_z$	$\|\mathbf{e}_z\|$	$\cos(\mathbf{e}_q, \mathbf{e}_z)$	Rank
z_1	$0.8(0.9) + 0.6(0.4) + 0 = 0.96$	0.985	**0.975**	1
z_2	$0.8(0.1) + 0.6(0.9) + 0 = 0.62$	0.911	0.681	3
z_3	$0.8(0.0) + 0.6(0.1) + 0 = 0.06$	1.005	0.060	4
z_4	$0.8(0.7) + 0.6(0.2) + 0 = 0.68$	0.735	**0.925**	2

Step 5: Top-k retrieval. With $k = 2$:

$$Z_2 = \underset{z \in \mathcal{D}}{\text{top-2}}\, \cos(\mathbf{e}_q, \mathbf{e}_z) = \{z_1, z_4\}. \tag{17}$$

The pizza chunk z_3 is correctly ignored. Brute-force search costs $O(|\mathcal{D}| \cdot d)$; FAISS/HNSW indexes make it sub-linear for millions of chunks.

Step 6: Build the prompt.

$$\text{prompt}(x, Z_2) = [\,\text{instruction};\ z_1;\ z_4;\ x\,] \tag{18}$$

```
Answer the question using only the context below.
If the answer is not in the context, say you don't know.

Context:
[1] The Machine Learning exam takes place on 15 February.
[2] For the exam, one handwritten A4 cheat sheet is allowed.

Question:  When is the Machine Learning exam?
```

Step 7: Generate. The frozen LLM decodes token by token (greedy decoding shown):

$$y_i = \arg\max_{v \in \mathcal{V}} p_\theta\big(v \mid \text{prompt}(x, Z_2),\, y_{1:i-1}\big), \qquad i = 1, 2, \ldots \tag{19}$$

Output: $y =$ "The exam is on 15 February [1]."

6 Worked Example: Original RAG with the Same Data

Retriever probabilities. DPR scores are raw inner products of 768-dimensional vectors, so they are large. To mimic that, scale the cosines from Section 5 by 10 and apply a softmax over the top-2:

$$s_{z_1} = 9.75, \quad s_{z_4} = 9.25, \qquad p_\eta(z_1 \mid x) = \frac{e^{9.75}}{e^{9.75} + e^{9.25}} = 0.621, \quad p_\eta(z_4 \mid x) = 0.379. \tag{20}$$

Generator probabilities. Let the target answer be $y = (y_1, y_2) =$ ("15", "February"). Assume the generator gives:

Document	$p_\theta(y_1 \mid x, z)$	$p_\theta(y_2 \mid x, z, y_1)$	$p_\theta(y \mid x, z) =$ product
z_1 (exam date)	0.90	0.95	0.855
z_4 (cheat sheet)	0.40	0.70	0.280

z_4 mentions the exam but not the date, so the generator is less sure with it.

RAG-Sequence, Eq. (2).

$$p(y \mid x) = 0.621 \times 0.855 + 0.379 \times 0.280 = 0.531 + 0.106 = 0.637. \tag{21}$$

RAG-Token, Eq. (3). Mix per token first, then multiply:

$$p(y_1 \mid x) = 0.621 \times 0.90 + 0.379 \times 0.40 = 0.711, \tag{22}$$

$$p(y_2 \mid x, y_1) = 0.621 \times 0.95 + 0.379 \times 0.70 = 0.855, \tag{23}$$

$$p(y \mid x) = 0.711 \times 0.855 = 0.608. \tag{24}$$

Loss. For RAG-Sequence, Eq. (4) for this single example is

$$\mathcal{L} = -\log 0.637 = 0.451. \tag{25}$$

Gradient for the retriever. Write $p(y \mid x) = \sum_z \mathrm{softmax}(s)_z \, g_z$ with $g_z = p_\theta(y \mid x, z)$. Differentiating $-\log p(y \mid x)$ with respect to a score gives

$$\frac{\partial \mathcal{L}}{\partial s_z} = \underbrace{p_\eta(z \mid x)}_{\text{prior}} - \underbrace{\frac{p_\eta(z \mid x)\, g_z}{\sum_{z'} p_\eta(z' \mid x)\, g_{z'}}}_{\text{posterior } p(z|x,y)}. \tag{26}$$

In the example the posteriors are $0.531/0.637 = 0.833$ for z_1 and 0.167 for z_4, so

$$\frac{\partial \mathcal{L}}{\partial s_{z_1}} = 0.621 - 0.833 = -0.212, \qquad \frac{\partial \mathcal{L}}{\partial s_{z_4}} = 0.379 - 0.167 = +0.212. \tag{27}$$

Gradient descent therefore *raises* the score of z_1 and lowers z_4. The retriever learns to prefer documents that help the generator produce the correct answer, without ever being told which document is "correct". This gradient flows into BERT_q through $s_z = \mathbf{d}(z)^\top \mathbf{q}(x)$. This is the step modern RAG skips.

References

1. P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. `https://arxiv.org/abs/2005.11401`
2. V. Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020. `https://arxiv.org/abs/2004.04906`
3. O. Ram et al. In-Context Retrieval-Augmented Language Models. TACL 2023. `https://arxiv.org/abs/2302.00083`
4. W. Shi et al. REPLUG: Retrieval-Augmented Black-Box Language Models. 2023. `https://arxiv.org/abs/2301.12652`
5. G. Izacard and E. Grave. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering (FiD). 2020. `https://arxiv.org/abs/2007.01282`
6. Y. Gao et al. Retrieval-Augmented Generation for Large Language Models: A Survey. 2023. `https://arxiv.org/abs/2312.10997`
7. E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. 2021. `https://arxiv.org/abs/2106.09685`